COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	July 28, 2026	**DATE SUBMITTED:**	August 1, 2026
NAME:	Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK NO. 4.1 STACKS

1. PROCEDURE OUTPUT

What procedures did you perform?

ILOA: FIRST PART

main.cpp

C:\Users\TIPQC\Documents\HOA4p1_REYES\ILOA\main.cpp

```cpp
#include <iostream>
#include <stack> // call the stack from the STL


int main(){
    std::stack<int> newStack; // created an object

    // testing the empty() function to check if te stack is empty
    std::cout<<"Is the stack empty? " << newStack.empty()<<std::endl;

    //insert a 3 elements:
    newStack.push(3);
    std::cout<<"Top of stack: " << newStack.top()<<std::endl;
    newStack.push(2);
    std::cout<<"Top of stack: " << newStack.top()<<std::endl;
    newStack.push(1);
    std::cout<<"Top of stack: " << newStack.top()<<std::endl;

    std::cout<<"Is the stack empty? " << newStack.empty()<<std::endl;
    newStack.push(6);
    newStack.push(7);
    newStack.push(8);
    std::cout<<"Size of the stack is: " << newStack.size()<<std::endl;

    // first part ends here

    return 0;
}
```

ILOA: SECOND PART

main.cpp ILOA | main.cpp ILOB

ILOA > main.cpp > main()

```cpp
#include <iostream>
#include <stack> // call the stack from the STL

int main(){
    std::stack<int> newStack; // created an object
    std::stack<int> stack1; // for experiment ~ swapping
    std::stack<int> stack2; // for experiment ~ swapping

    // testing the empty() function to check if the stack is empty
    std::cout << "Is the stack empty? " << newStack.empty() << std::endl;

    // insert 3 elements:
    newStack.push(3);
    std::cout << "Top of stack: " << newStack.top() << std::endl;
    newStack.push(2);
    std::cout << "Top of stack: " << newStack.top() << std::endl;
    newStack.push(1);
    std::cout << "Top of stack: " << newStack.top() << std::endl;

    // SIZE OF THE STACK
    std::cout << "Is the stack empty? " << newStack.empty() << std::endl;
    newStack.push(6);
    newStack.push(7);
    newStack.push(8);
    std::cout << "Size of the stack is: " << newStack.size() << std::endl;

    // first part ends here

    // swap? -> swap the content of the public member functions
    stack1.push(100); // for experiment ~ swapping
    stack2.push(200); // for experiment ~ swapping

    stack1.swap(stack2); // for experiment ~ swapping

    std::cout << "Stack 1 after swap: " << stack1.top() << std::endl; // Should print 200
    std::cout << "Stack 2 after swap: " << stack2.top() << std::endl; // Should print 100

    // emplace -> construct and insert element -> public member function
    // Example usage of emplace:
    stack1.emplace(300);
    std::cout << "Stack 1 after emplace: " << stack1.top() << std::endl; // Should print 300

    return 0;
}
```

```
PS C:\Users\TIPQC\Documents\HOA4p1_REYES>  & 'c:\Users\TIPQC\.vscode\extensions\ms-vs
m4lbneq.5ei' '--stdout=Microsoft-MIEngine-Out-ok02wxvk.i3w' '--stderr=Microsoft-MIEng
 '--interpreter=mi'
Is the stack empty? 1
Top of stack: 3
Top of stack: 2
Top of stack: 1
Is the stack empty? 0
Size of the stack is: 6
Stack 1 after swap: 200
Stack 2 after swap: 100
Stack 1 after emplace: 300
PS C:\Users\TIPQC\Documents\HOA4p1_REYES>
```

CODE ANALYSIS FOR ILOA:

Going through these procedures and exercises helped me gain a better understanding of how stacking works in C++ language. With concepts like pushing, popping, checking the state–"top," "isempty," "isfull" it paved the way for me to understand how fundamentals in stacking truly works. With the given code, first thing that was done is that it includes the "#include <stack" in order to validate all of the stacking that will be done all throughout the code. From line 5 to 7, it first starts off with creating an object "newstack," "stack1," "stack2" in order for the IDE to recognize these. Then in line 9, it just basically prints and checks if the stack is empty by calling "newStack.empty()". From line 12 to 18, the thing that was done is that we use the push function to enter our desired number to be inputted at the top of the stack. Then, we print it off. After doing so, we checked the size of the stack by calling "newStack.size()" which displays 6. Furthermore, we also explored swapping in this type of topic. Basically, we just created a new object below the main function to validate, and then we push it at the bottom, and use the swapping function, then print. At last, in emplace, we just simply created a new value and we just put it off at the head of our stack. Overall, this activity helped me better understand the concept of stacking by showing how it really works in the coding language. Also, the reason why I think I better grasp the topic now is that the visual learning system is much clearer and comprehensible to me.

ILOB: PART 1:

main.cpp ILOA | main.cpp ILOB

ILOB > main.cpp > isEmpty()

```cpp
#include <iostream>

const size_t maxCap = 100; // maximum capacity of the size
int stack [maxCap];
int top = -1, newData, elements;

// prototyping
bool isEmpty();
bool isFull();
void push();
void pop();
void Top();

int main(){
    int choice;

    // ask user for the maximum elements to be inserted
    std::cout<<"Enter the number of maximum elements for the new stack (max: 100): ";
    std::cin >> elements;


    while(true){
        std::cout<<"Stack Operations: "<<std::endl;
        std::cout<<"1. PUSH \n 2. POP \n 3. TOP \n 4. isEmpty \n 5. isFull"<<std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<< isEmpty()<<std::endl;
            break;
            case 5: std::cout<< isFull()<< std::endl;
            default: std::cout<<"invalid choice" <<std::endl;
            break;
        }

    }

    return 0;
}

bool isEmpty(){
    // top is -1
    if (top == -1) return true;
    std::cout<<"Stack Underflow";
    return false;

}
```

main.cpp ILOA | main.cpp ILOB

ILOB > main.cpp > isEmpty()

```
46  bool isEmpty(){
53
54  bool isFull(){
55      // not equal to the maxCap
56      if(top >= elements){
57          std::cout<<"Stack Overflow";
58          return true;
59      }
60      return false;
61  }
62
63  void push(){
64
65      // check if the stack is full, if yes return an error
66      if(isFull()) return;
67
68      // ask the user for a value to be pushed into the stack
69      std::cout<<"What member do you want to add to the stack? ";
70      std::cin>> newData;
71
72      // store inside the stack?
73      stack[++top] = newData;
74
75  }
76
77  void pop(){
78
79      // check if the stack is empty / there's no more elements inside the stack
80      if(isEmpty()) return;
81
82      // display the elements to be popped:
83      std::cout<<"Popping: " <<stack[top];
84
85          // decrement top?
86          top--;
87
88      }
89
90      void Top(){
91          // error checking?
92          if(isEmpty()) return;
93
94          // display the top element
95          std::cout<<"The top element is: " << stack[top]<<std::endl;
96
97      }
```

ILOB1 PART 2: ADDING DISPLAY ALL TO THE SYSTEM

main.cpp ILOA | main.cpp ILOB

ILOB > main.cpp > Top()

```cpp
#include <iostream>

const size_t maxCap = 100; // maximum capacity of the size
int stack [maxCap];
int top = -1, newData, elements;

// prototyping
bool isEmpty();
bool isFull();
void push();
void pop();
void Top();
void displayAll(); // displayAll();

int main(){
    int choice;

    // ask user for the maximum elements to be inserted
    std::cout<<"Enter the number of maximum elements for the new stack (max: 100): ";
    std::cin >> elements;


    while(true){
        std::cout<<"Stack Operations: "<<std::endl;
        std::cout<<"1. PUSH \n 2. POP \n 3. TOP \n 4. isEmpty \n 5. isFull \n 6. displayAll" <<std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<< isEmpty()<<std::endl;
            break;
            case 5: std::cout<< isFull()<< std::endl;
            break;
```

main.cpp ILOA | main.cpp ILOB

ILOB > main.cpp > Top()

```cpp
int main(){
    while(true){
        switch(choice){
            break;
            case 6: displayAll();
            break;
            default: std::cout<<"invalid choice" <<std::endl;
            break;
        }

    }

    return 0;
}

bool isEmpty(){
    // top is -12
    if (top == -1) return true;
        std::cout<<std::endl;
        return false;

}

bool isFull(){
    // not equal to the maxCap
    if(top >= elements){
        std::cout<<"Stack Overflow \n";
        return true;
    }
    return false;
}
```

main.cpp ILOA | main.cpp ILOB

ILOB > main.cpp > Top()

```
58  bool isFull(){
65  }
66
67  void push(){
68
69      // check if the stack is full, if yes return an error
70      if(isFull()) return;
71
72      // ask the user for a value to be pushed into the stack
73      std::cout<<"What member do you want to add to the stack? ";
74      std::cin>> newData;
75
76      // store inside the stack?
77      stack[++top] = newData;
78
79  }
80
81  void pop(){
82
83      // check if the stack is empty / there's no more elements inside the stack
84      if(isEmpty()) return;
85
86      // display the elements to be popped:
87      std::cout<<"Popping: " <<stack[top];
88
89      // decrement top?
90      top--;
91
92  }
93
```

```
94  void Top(){
95      // error checking?
96      if(isEmpty()) return;
97
98      // display the top element
99      std::cout<<"The top element is: " << stack[top]<<std::endl;
100
101 }
102
103 void displayAll() {
104     // Check if the stack has any elements to display
105     if(isEmpty()) return;
106
107     for (int i = top; i >= 0; i--) {
108         std::cout << stack[i]<<std::endl;
109     }
110     std::cout << std::endl;
111 }
```

```
PS C:\Users\TIPQC\Documents\HOA4p1_REYES>  & 'c:\Users\TIPQC\.v
5acrity.ris' '--stdout=Microsoft-MIEngine-Out-v0xipajm.43g' '--
 '--interpreter=mi'
What member do you want to add to the stack? 1
Stack Operations:
1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. isFull
 6. displayAll
1
What member do you want to add to the stack? 2
Stack Operations:
1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. isFull
 6. displayAll
1
What member do you want to add to the stack? 3
Stack Operations:
1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. isFull
 6. displayAll
6

3
2
1

Stack Operations:
1. PUSH
 2. POP
 3. TOP
 4. isEmpty
 5. isFull
 6. displayAll
```

CODE ANALYSIS ILOB:

This implementation in C++ language demonstrates a fundamental Array-Based Stack Data Structure, which is a critical concept used in order to manage memory, manage undo/redo operations, and parsing syntax. By tracking a top pointer within a fixed-sized memory block, the program highlighted the concept our professor was teaching which is the Last-In, First-Out (LIFO) data processing principle. Doing this exercise reveals the importance of proactive error handling, specifically in managing Stack Overflow which adds data to a full storage/ Stack Underflow which removes data from an empty one. By utilizing an index variable initialized to -1, the system tracks and controls active data positioning within the allocated stack memory which is given here in this code is 100. Furthermore, using isFull() and isEmpty() conditions, paves the way to eliminate risks of memory corruption via Stack Overflow or Stack Underflow. Overall, this code helped me better understand the concept of Stacking into a little deeper than the previous procedure part which I think I still need some time to practice in order for me to master and write the code in an optimized level.

ILOB2: CONTINUATION

main.cpp | listStack.h

listStack.h > ...

```
1   // header file for listStack.h file
2   #ifndef LIST_STACK_H
3   #define LIST_STACK_H
4   #include <iostream>
5
6   // create the class node
7   template <typename T>
8   class Node{
9       public:
10          T data;
11          Node<T> *next = nullptr;
12  };
13
14  // create a pointer/reference top that always points to the current top node of the stack
15
16  // global
17  template <typename T>
18  Node<T>* head = nullptr;
19
20  // pushing operation:
21  template <typename T>
22  void push(T newData){
23      // create a new node
24      Node<T>* newNode = new Node<T>;
25      // assign the data to the new node
26      newNode->data = newData;
27      // point the newNode next to the current top
28      newNode->next = head<T>;
29      // assign the new Top to the newNode
```

main.cpp | listStack.h

listStack.h > ...

```
2     #ifndef LIST_STACK_H
22    void push(T newData){
30        head<T> = newNode;
31    }
32
```

CODE ANALYSIS:

This part of the code creates the basic structure of a stack using a linked list in C++. It defines a Node class to store data and connect each node, while the head pointer always keeps track of the top of the stack. The push() function adds a new element by creating a node, linking it to the current top, and updating the head pointer. This shows how a linked-list stack can grow dynamically as new elements are added.

```
33  // isEmpty() operation
34  template <typename T>
35  bool isEmpty(){
36      return head<T> == nullptr;
37
38  }
39
40  template <typename T>
41  T pop(){
42      // if the stack is empty error catch:
43      if(isEmpty<T>()){
44          std::cout<<"Stack underflow" << std::endl;
45      }
46
47      // saving a current top node to a temporary variable
48      Node<T>* temp = head<T>;
49      // store the data to a temp variable
50      T tempVal = temp->data;
51      // move the head to the next node
52      head<T> = head<T>->next;
53      // free up the memory
54      delete temp;
55      // return the temporary variable data
56      return tempVal;
57  }
```

CODE ANALYSIS:

This part of the code handles checking if the stack is empty and removing the top element. The isEmpty() function simply checks if the head pointer is nullptr, while the pop() function removes the top node, returns its value, and frees the memory it used. It also includes a basic stack underflow check to notify the user when attempting to remove an element from an empty stack.

main.cpp | listStack.h

listStack.h > ...

```
2   #ifndef LIST_STACK_H
58
59  template <typename T>
60  void Top(){
61      // what if the stack is empty??
62      if(isEmpty<T>()) return;
63      std::cout<<"The top is: " << head<T>->data<<std::endl;
64  }
65
66  // Display all
67  template <typename T>
68  void displayAll(){
69      // checks immediately if the valued return is empty
70      if(isEmpty<T>()) return;
71      // store the head with temp
72      Node<T> *temp = head<T>;
73
74      while(head<T> != nullptr){
75          std::cout<< head<T>->data<<" ";
76          head<T> = head<T>->next;
77      }
78      // to check if the head doesn't change its value
79      head<T> = temp;
80      std::cout<<std::endl;
81  }
82
83
84  #endif
```

CODE ANALYSIS:

This part of the code provides functions to view the top element and display all the elements in the stack. The Top() function prints the value at the top only if the stack is not empty, while displayAll() traverses the linked list and displays each element from top to bottom. It temporarily saves the head pointer to ensure the original stack structure remains unchanged after displaying the contents.

CODE OUTPUT ILOB2:

```
[Running] cd "c:\Users\TIPQC\Documents\ILOB2\" && g++ main.cpp
===== Pushing 'I' 'E' 'E' 'E' =====

The top is: E
Display All Values: E E E I

===== Pushing 'E' 'E' 'E' 'I' =====
Popping: E
Popping: E
Popping: E
Popping: I

Stack underflow
```

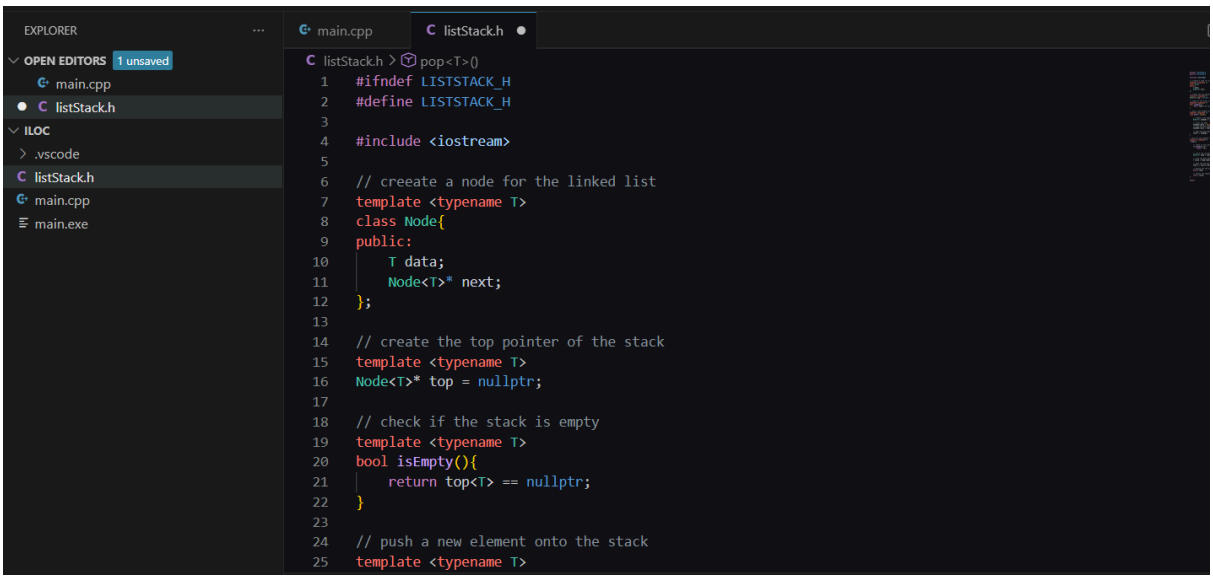
CODE ANALYSIS ILOB2:

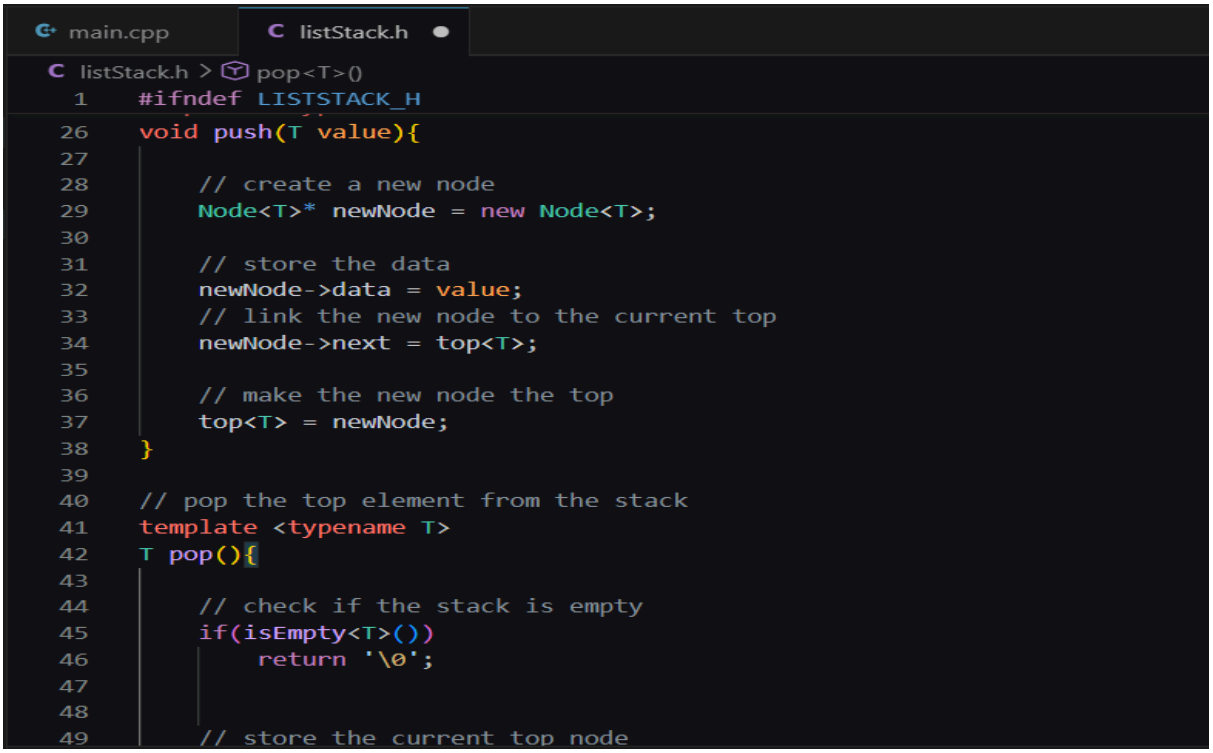
These blocks of code which are being used are similar to the previous code being implemented that highlights the use of array based, but the only difference this time is that the standard template library (STL) is being utilized. At the start of the code, we created two files which are the main.cpp and listStack.h. With the first file, “listStack.h,” we use global head pointer for a template class, which breaks for multiple types and ruins the stack state during display. At the main file, we recognized at the header section the listStack.h to connect it to the main file, and here, we start to push variables, display all the values, pop and print elements, and we also test the stack underflow.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILOC: CODE INPUT
(listStack.h)





```
50      Node<T>* temp = top<T>;
51
52      // save the data before deleting
53      T value = temp->data;
54
55      // move the the top to the next node
56      top<T> = top<T>->next;
57
58      // delete the old top node
59      delete temp;
60
61      // return the removed value
62      return value;
63  }
64
65  #endif
```

ILOC: CODE INPUT
(main.cpp)

main.cpp × | listStack.h ●

main.cpp > main()

```
1   #include <iostream>
2   #include <string>
3   using namespace std;
4
5   const int MAX = 100;
6
7   // stack class using an array
8   class Stack{
9   private:
10      char arr[MAX];
11      int top;
12
13  public:
14      // initializes an empty stack
15      Stack(){
16          top = -1;
17      }
18
19      // check if the stack is empty
20      bool isEmpty(){
21          return top == -1;
22      }
23
24      // push an opening symbol onto the stack
25      void push(char c){
```

main.cpp | listStack.h

main.cpp > main()

```
8   class Stack{
25      void push(char c){
26          arr[++top] = c;
27      }
28
29      // pop the top symbol from the stack
30      char pop(){
31          if (isEmpty())
32              return '\0';
33
34          return arr[top--];
35      }
36  };
37
38  // check if opening and closing symbols match
39  bool match(char open, char close){
40      return (open == '(' && close == ')') ||
41             (open == '[' && close == ']') ||
42             (open == '{' && close == '}');
43  }
44
45  // check whether the expression is balanced
46  bool checkBalance(string exp){
47      // create a stack
48      Stack s;
```

main.cpp | listStack.h

main.cpp > main()

```
46  bool checkBalance(string exp){
49
50      // read each character
51      for(char ch : exp){
52          // ignore non-symbol characters
53
54          // push opening symbols
55          if(ch=='('||ch=='['||ch=='{'){
56              s.push(ch);
57          }
58          // process closing symbols
59          else if(ch==')'||ch==']'||ch=='}'){
60              // report error if stack is empty
61              if(s.isEmpty())
62                  return false;
63
64              // pop the top opening symbol
65              char temp = s.pop();
66
67              // compare opening and closing symbols
68              if(!match(temp,ch))
69                  return false;
70          }
71      }
72      // stack must be empty
```

```
73      return s.isEmpty();
74  }
75
76  int main(){
77      string expression;
78
79      // ask the user for an expression
80      cout<<"Enter expression: ";
81      getline(cin,expression);
82
83      // display the result
84      if(checkBalance(expression))
85          cout<<"Expression is Balanced."<<endl;
86      else
87          cout<<"Expression is Not Balanced."<<endl;
88
89      return 0;
90  }
```

CODE OUTPUT:

```
PS C:\Users\Angel Elayza Reyes\OneDrive\Documents\ILOC> & "C:\Users\Angel Elayza Reyes\OneDrive\Documents\ILOC\main.exe"
Enter expression: (A+B)+(C-D)
Expression is Balanced.
PS C:\Users\Angel Elayza Reyes\OneDrive\Documents\ILOC> & "C:\Users\Angel Elayza Reyes\OneDrive\Documents\ILOC\main.exe"
Enter expression: ((A+)+(C-D)
Expression is Not Balanced.
PS C:\Users\Angel Elayza Reyes\OneDrive\Documents\ILOC> & "C:\Users\Angel Elayza Reyes\OneDrive\Documents\ILOC\main.exe"
```

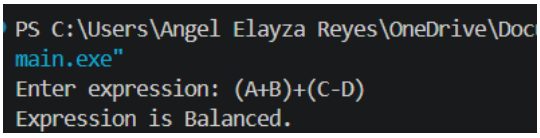
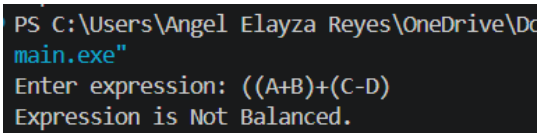
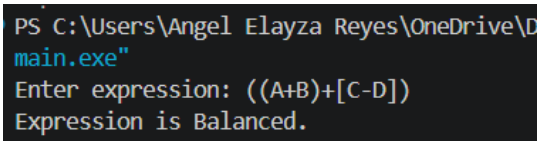
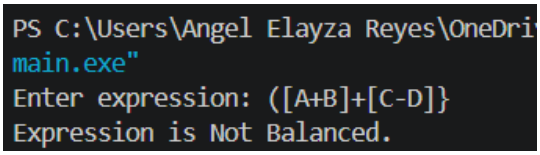
```
PS C:\Users\Angel Elayza Reyes\OneDrive\Documents\ILOC> & "C:\Users\Angel Elayza Reyes\OneDrive\Documents\ILOC\main.exe"
Enter expression: ((A+B)+[C-D])
Expression is Balanced.
```

```
PS C:\Users\Angel Elayza Reyes\OneDrive\Documents\ILOC> & "C:\Users\Angel Elayza Reyes\OneDrive\Documents\ILOC\main.exe"
Enter expression: ([A+B]+[C-D]}
Expression is Not Balanced.
```

COMPUTER ENGINEERING

QUEZON CITY

Self-Checking:
For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		Every opening parenthesis has a corresponding closing parenthesis. The stack is empty after processing the expression, indicating that all delimiters are properly matched.
((A+B)+(C-D)	N		One opening parenthesis remains unmatched at the end of the expression. Since the stack is not empty after processing all characters, the expression is considered unbalanced.
((A+B)+[C-D])	Y		The parentheses () and square brackets [] are correctly paired and properly nested. Every opening delimiter matches its corresponding closing delimiter, leaving the stack empty.
((A+B]+[C-D]}	N		The expression ends with a closing curly brace } instead of the expected closing parenthesis). The popped opening delimiter does not match the closing delimiter, causing the expression to be unbalanced.

CONCLUSION AND CODE ANALYSIS:

CODE ANALYSIS:

The method used in this supplementary part illustrates how the data structure known as a stack can be used to determine whether an expression represents balanced characters such as brackets ( ), square brackets [ ], curly brackets { }. In the method, two implementations of the stack are shown. The first implementation uses an array while the second one uses linked lists. As the program processes the whole expression character by character, it neglects letters and mathematical operators, pushes all open brackets to the stack, and removes the top element of the stack at the moment a closed symbol is met. The symbol removed from the stack is then compared with the closed symbol to check whether it is a match. If a comparison fails or the stack is empty when the closing symbol is met, the expression is considered not balanced. After processing all the symbols, it is necessary for the stack to be empty so that the expression can be recognized as balanced.

In terms of implementation, a stack can be implemented by using either an array or a linked structure. The array implementation uses a fixed array of predetermined size to store the stack contents, which is easy and efficient for the expressions that fall in the range specified by the size of the array. The second way of implementing a stack is through a linked list, which is composed of dynamically allocated nodes in which each element is stored. A linked implementation is beneficial since it allows for a variable size of the stack according to the needs of the user without requiring fixed-size implementation of the stack. The two ways of stack implementation yield the same results; however, they differ in their memory use.

To sum it all up, this hands-on activity demonstrates how crucial stacks can be in dealing with real-world programming issues, especially considering the necessity of checking whether symbols in expressions are balanced. Working with the stacks based on arrays and linked lists made it easier to realize what the Last-In, First-Out principle means and why it is fit for matching the opening and closing symbols. It was also observed that although different types of implementations may have mixed memory management solutions, they still achieve the same results. On the whole, this exercise enhanced my understanding of the topic of stacks and their usage in such applications as compilers, syntax checks, and expression validation.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance, Code Error Explanation, Grammar Correction, and Explanation of Concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: AI only paved the way for me to correct my mistakes and errors as well as explain the concept of stacking in comprehensible form which I can visualize.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.